Bags, Vectors & Transformers

From PDFs (and Other Files) to Analyzable Text · Bonus, Part 1

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

The Hidden First Step

Every method in this course starts from clean text in a table. Real research material rarely arrives that way:



PDF reports
policy briefs, annual reports

Scans
archives, old newspapers



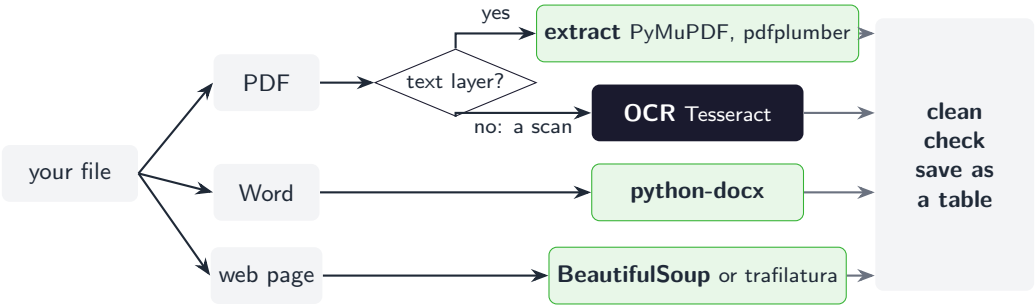
Word files
interview transcripts



Web pages
news, party websites

Getting from files to text is often the most time-consuming part of a project, and its errors are **silent**: a page header counted as words in every document, a scrambled two-column layout, a scan read as gibberish. Everything downstream inherits them.

One Question Decides the Method



Does the PDF have a text layer? If you can select and copy text in your PDF viewer, probably yes. If extraction returns almost nothing, it's a scan and needs OCR. That test is easy to automate.

What Goes Wrong with PDFs

A PDF doesn't store paragraphs or columns, only *"draw these characters at these coordinates"*. Reconstructing the reading order causes most of the trouble.

Two columns, read straight across the page: every line glues two unrelated halves

```
Agricultural nitrogen emissions have become one This brief summarizes recent research on three
of the most contested policy issues in the Netherl- questions.  First, how large are the health effects
```

Headers, page numbers

```
POLICY BRIEF | No.  7
Page 1 of 2
```

Hyphenation

```
...  threaten their livel-
ihoods and the ...
```

Ligatures, tables

"fi" is one special character, so *financial* $\neq$ *financial*. Tables become loose words.

Three Python Libraries for Born-Digital PDFs

pypdf

· pure Python, simplest to use

· follows the order in which text was written into the file

· fine for simple, one-column documents

pdfplumber

· knows the position of every character and word

· **cropping**: read only part of a page

· **table extraction** into pandas

· by far the slowest

PyMuPDF

· very fast (a C library inside)

· text as layout **blocks** with positions

· renders pages as images, for OCR

· the workhorse for large collections

No reading-order setting is right for every PDF. In the notebook, sorting blocks "top to bottom" scrambles a two-column page, while the file order happens to be right. Always look at the output.

Scanned Documents: OCR with Tesseract

1. **Render** each page as an image, at about 300 dpi
2. Optionally **clean up** the image: grayscale, threshold, straighten
3. Run **Tesseract** with the right **language model**
4. Use the per-word **confidence** scores to target manual checks

Slow (seconds per page, vs. milliseconds for a text layer) and never perfect, so measure the error rate on a sample.

The language model matters

`lang="eng"` on a Dutch scan:
De co**é**rdinatie ... een efficié**n**t stikstofbeleid

`lang="nld"`:
De co**ö**rdinatie ... een efficië**n**t stikstofbeleid

Output from the notebook. Tesseract supports 100+ languages; combine them with `lang="nld+eng"`.

Measuring Extraction Quality

The **character error rate** (CER): the minimum number of characters to insert, delete or change to turn the extracted text into the true text, divided by the length of the true text.

kitten → *sitting*: 3 edits
CER 0.02 = 2 errors per 100 characters

Since the notebook generates its own documents, it knows the true text and can measure every method exactly. With your own files: hand-type a few pages as the reference.

Results on the notebook's test brief

method	CER
pypdf, whole pages	0.196
pdfplumber, whole pages (columns glued)	0.707
pdfplumber, cropped to the columns	0.004
+ cleaning	0.000
OCR of the scan, cropped + cleaned (page 1)	0.003

Whole-page extractions include headers, page numbers and the table, which count as errors. In a word count, they *are* noise.

Cleaning, and a Pipeline for Whole Folders

A conservative cleaning function

```
# ligatures, odd spaces
text = unicodedata.normalize("NFKC", text)
# re-join hyphenated words
text = re.sub(r"(\w)-\n(\w)", r"\1\2", text)
# single line break -> space
text = re.sub(r"(?<!\n)\n(?!\n)", " ", text)
```

Each rule can also do damage: the hyphen rule turns *evidence-based* at a line end into *evidencebased*. Keep the raw text too.

From a folder to a dataset

file	page	method	text
brief.pdf	1	text layer	Nitrogen …
scan.pdf	1	OCR	Nitrogen …
notes.docx		docx	Interview …
news.html		html	Government …

Record the method for every row, so OCR rows can be checked separately, and **never let one bad file stop the run**: log the error and move on.

Harder Cases, and a Word on Privacy

problem	tools worth knowing
complex layouts, tables, Markdown output	Docling, Marker, PyMuPDF4LLM
scientific articles	GROBID
poor scans, unusual scripts	EasyOCR, PaddleOCR
handwriting, historical documents	Transkribus
very messy pages	vision-language models

Vision-language models read difficult pages well, but they can **hallucinate**: fluent text that isn't on the page, which classic OCR never does.

Privacy: transcripts and internal reports are often confidential. Online tools (and Colab) run on someone else's servers. Check your data management plan; everything in the notebook also runs locally.

Checklist: look at a sample first · choose a method per file type · record the method · hand-check a random sample · clean conservatively · report the process and its error rate

In the Notebook

- Generating test documents: a two-column brief, scans, a Word file, a web page
- pypdf vs. pdfplumber vs. PyMuPDF, cropping, and tables into pandas
- Measuring extraction quality with the character error rate
- Cleaning: hyphenation, line breaks, ligatures, running headers
- OCR with Tesseract, in English and Dutch, with confidence scores
- A batch pipeline from a folder of mixed files to one dataset

`bonus/notebooks/1_from_pdfs_to_text_exercises.ipynb` · 5 exercises · no GPU needed